STM32 TFT Designer

Version 1.40

User Manual

by Carlos Barberis

© 2026 Carlos Barberis. All rights reserved.

Table of Contents

1. Introduction

STM32 TFT Designer is a desktop application for laying out graphical user interfaces for SPI TFT displays driven by an STM32 microcontroller. Instead of hand-placing pixel coordinates and writing repetitive draw calls, you arrange buttons, labels, sliders, plots, and other widgets visually on a virtual screen, then export ready-to-compile C source and header files for your STM32CubeIDE project.

The application targets a small family of common TFT controllers, including ILI9341, ST7796, ST7789, and ST7735, and produces code intended to sit on top of a tft_gfx / tft_widgets / tft_graph driver layer.

The exported code is plain C with no external dependencies beyond the driver layer already used in your project, so the generated files can be dropped directly into Core/Src and Core/Inc.

> *Scope: STM32 TFT Designer is built specifically for STM32 microcontroller projects developed in STM32CubeIDE. It is not a general-purpose UI designer for other microcontroller families (such as ESP32, PIC, AVR, or other Arm Cortex-M vendors), and the generated code assumes the STM32 HAL/CubeMX project structure (Core/Src, Core/Inc) and a tft_gfx / tft_widgets / tft_graph driver layer written for STM32. Porting the generated output to a different toolchain or microcontroller family is possible in principle, since the generated code is plain C, but is not a supported or tested workflow.*

This manual covers the use of the designer application itself: the workspace, widgets, properties, layout tools, and the export and integration workflow.

2. Quick Start

This section gets a simple screen from blank canvas to exported C code in a few minutes.

1. Launch the application. A new project opens automatically with one screen named main and a default display preset selected.
2. In the Display panel on the left, choose the TFT controller and resolution that matches your hardware (for example, ILI9341 320x240), and set the Rotation if your display is mounted other than the default orientation.
3. In the Widgets panel, click Text Label to add a label to the canvas. It appears at a default position; drag it where you want it.
4. Click the label to select it, then use the Properties / Code panel on the right to change its text, position, size, and colors.
5. Add a Button and a Dynamic Text widget the same way, and set their properties.
6. Use File → Export ui_screens.c/.h to choose an output folder. The designer writes ui_screens.c, ui_screens.h, ui_display_config.h, and a runtime notes file into that folder.
7. Copy the generated .c files into your CubeIDE project's Core/Src folder and the .h files into Core/Inc, then call UI_Init() and UI_DrawScreen() from your firmware as described in Section 14.

3. Installation and Requirements

STM32 TFT Designer runs on Windows and macOS (including Apple Silicon). It is built with Python and PySide6 (Qt for Python), and optionally uses Pillow for more reliable image loading when importing JPEG, BMP, and PNG logos.

If you are running the program from a packaged installer (.exe on Windows or .app on macOS), no additional setup is required. If you are running it from source, you will need:

- Python 3.10 or later
- PySide6
- Pillow (optional, but recommended; without it, only image formats supported by your installed Qt image plugins can be imported)

No internet connection is required to run the designer or to export code. The application does not transmit project data anywhere; all files are read from and written to locations you explicitly choose.

4. The Workspace

The main window is divided into three regions:

Canvas (center). A scrollable, zoomable graphics view representing the physical display. Widgets are placed and dragged here. The canvas always reflects the display resolution and rotation currently selected in the Display panel.

Project / Add Widgets (left dock). Contains the Display settings (controller preset, rotation, preview zoom, snap grid), the Screens list, and buttons to add each widget type to the current screen.

Properties / Code (right dock). Shows the editable properties of whichever widget is currently selected on the canvas, along with layout tools (alignment, distribution, layering, off-screen widget management) and a live code preview.

The menu bar at the top provides File (New, Open JSON, Save JSON, Export), Edit (Copy Widget, Paste Widget, Duplicate Widget, Delete Widget), and Help (About) menus.

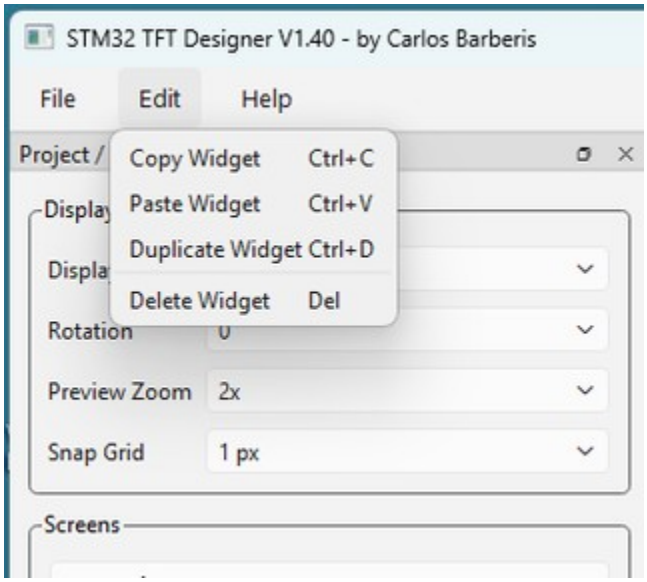
Edit Menu

The Edit menu (new in V1.40) provides clipboard and widget management operations that speed up layout work when you need to reuse or remove widgets:

- Copy Widget (Ctrl+C) — copies all currently selected widgets to an internal clipboard, preserving every property including position, size, colors, source variable bindings, and image paths. The clipboard is internal to the designer and does not interact with the operating system clipboard.
- Paste Widget (Ctrl+V) — places a copy of each widget on the clipboard onto the active screen, offset 15 pixels right and 15 pixels down from the original position. Each pasted widget is automatically given a unique name so it does not conflict with the source. If the offset would place the widget outside the display bounds it is clamped back into view. The clipboard is not cleared after pasting, so the same widget can be pasted multiple times.

- Duplicate Widget (Ctrl+D) — a one-step shortcut that copies and immediately pastes the selected widget(s) without affecting the clipboard. The duplicate appears 15 pixels offset from the original and receives a unique name. This is the fastest way to create multiple similar widgets on the same screen.
- Delete Widget (Del) — removes the currently selected widget from the active screen permanently. There is no undo, so confirm you have the right widget selected before pressing Delete. To recover an accidentally deleted widget, close without saving and reopen the last saved project JSON file.

All four Edit operations also appear in the Edit menu in the menu bar, which is a convenient reference for their keyboard shortcuts.



The Edit menu (V1.40) showing Copy Widget, Paste Widget, Duplicate Widget, and Delete Widget with their keyboard shortcuts

> *macOS note: V1.40 also fixes a macOS-specific issue where the Help menu did not appear in the menu bar because Qt was silently relocating the About item into the system application menu. This is corrected in V1.40 and the Help menu now appears correctly on both Windows and macOS.*

5. Display and Project Settings

These settings live in the Display group box in the left-hand panel and apply to the whole project, not to an individual screen.

Display

Selects the physical panel and resolution. Supported presets include ILI9341 320×240, ST7796 480×320, ST7789 240×240 / 240×320 / 240×280, and ST7735 128×160 / 160×128 / 160×80. Changing this immediately resizes the canvas; widgets keep their existing X/Y/W/H values, so widgets near the edge of a smaller display may end up partially or fully off-screen (see Section 10).

Rotation

A value from 0 to 3, matching the rotation argument used by most SPI TFT driver libraries. Rotating swaps the effective width and height for resolutions where width and height differ, and the canvas updates to match.

Preview Zoom

Scales the on-screen canvas view (1x–5x) for easier editing on small displays such as the ST7735. This is a view-only setting; it has no effect on exported coordinates.

Snap Grid

Snaps widget movement and resizing to a pixel grid (off, or 1/2/4/5/8/10 px) to make aligning widgets by eye easier. Disable it when you need single-pixel placement.

6. Working with Screens

A project can contain multiple screens, each with its own independent set of widgets. Screens are useful for representing different pages or modes of your device's UI (for example, a main monitoring screen and a settings screen).

Add. Creates a new, empty screen and switches to it.

Delete. Removes the currently selected screen. At least one screen must always exist; the designer will refuse to delete the last remaining screen.

Screen selector (dropdown). Switches the canvas between screens. Each screen's widgets are only visible and editable while that screen is active.

Exported code represents each screen as a value in the UI_ScreenId_t enum, in the order the screens appear in the dropdown, and your firmware selects which one to draw by passing that enum value to UI_DrawScreen().

7. Widget Reference

Click any of the following buttons in the Widgets group to add that widget type to the current screen, centered or near the top-left of the canvas by default. The new widget is automatically selected so you can immediately edit its properties.

Widget	Purpose
Button	A static, labeled rectangular button intended for touch input that performs an action while pressed and released (tap behavior is defined in your touch-handling firmware, not the widget itself).
Momentary Button	Similar to Button, but intended for press-and-hold style controls where the active state matters while the button is held.
Checkbox	A toggle with an on/off visual state.
Radio Button	One of a mutually exclusive group; radio buttons added to the same screen are grouped automatically for code generation purposes.

Text Label	Static text that does not change at runtime, useful for titles, captions, and units.
Dynamic Text	A text field intended to be updated at runtime from a firmware variable, generating a typed UI_Set_...() function (see Section 13).
Progress Bar	A horizontal or vertical bar representing a value between a minimum and maximum.
Slider	A horizontal or vertical control with a draggable handle representing a value between a minimum and maximum.
Plot / Graph	A scrolling or sweeping data plot with configurable axes, gridlines, and update timing, intended for live sensor or measurement data.
Image / Logo Import	Imports a JPEG, BMP, or PNG image and converts it to an RGB565 pixel array embedded in the generated C source.
Rectangle	A simple filled or outlined rectangle, with optional rounded corners.
Circle / Ellipse	A simple filled or outlined circle, sized by diameter.

Every widget has a unique Name, used to generate C identifiers and function names, so it should contain only letters, numbers, and underscores, and should not start with a digit.

8. Editing Widget Properties

Selecting a widget on the canvas (click once) populates the Selected Widget group in the right-hand panel. The fields shown depend on the widget type; common ones include:

Geometry

X, Y, Width, and Height in pixels, relative to the top-left corner of the display in its current rotation. For the Circle widget, geometry is controlled by a single Diameter field instead of independent width and height.

Colors

Up to five color roles, each chosen from the same fixed palette offered in the dropdown (TFT_BLACK, TFT_WHITE, TFT_RED, TFT_GREEN, TFT_BLUE, TFT_CYAN, TFT_YELLOW, TFT_ORANGE, TFT_MAGENTA, TFT_DARKGREY, TFT_LIGHTGREY):

> *Note on TFT_LIGHTGREY: earlier copies of tft_colors.h did not define TFT_LIGHTGREY, even though it appeared in the designer's color dropdown, which caused an “undeclared identifier” build error for any widget using that color. The current tft_colors.h adds #define TFT_LIGHTGREY 0xD69A (a light grey, distinctly lighter than TFT_DARKGREY) to resolve this. If you're working from an older copy of tft_colors.h and hit this error, update the header to add that line, or substitute TFT_DARKGREY for the affected widget and re-export.*

- Text Color — foreground pixels for printed text.
- Text Background — the background drawn behind printed text characters.
- Box Fill — fill color for the widget's body or container area.

- Active/On/Fill Value — meaning depends on widget type (for example, the “on” state of a checkbox, or the filled portion of a progress bar).
- Border — outline color.

Text and Alignment

Editable display text, a text size multiplier (1–5, where each step is a multiple of the base MCU bitmap font size), and independent horizontal (left/center/right) and vertical (top/middle/bottom) alignment.

Dynamic Text Specifics

Max Value Chars reserves erase width for the printed value so old digits don't leave artifacts when a shorter value is printed later; Value Width (px) can override that reservation directly in pixels (0 means “calculate automatically from Max Value Chars”); Unit Spacing (px) controls the gap between the numeric value and its unit string.

Variable Binding (Dynamic Text, Slider, Progress Bar, Plot)

Source Variable names the C variable your firmware will provide; for Plot, Slider, and Progress Bar widgets this name is emitted verbatim into the generated header as an extern declaration, while Dynamic Text widgets do not get an extern and instead expect the value pushed in through a generated setter function. Source Type (float, uint16_t, uint32_t, int32_t, string, or expression) determines the C type used for the extern (where applicable) and the Dynamic Text setter's parameter type; Source Format controls the printf-style format used to render numeric values as text.

Plot / Graph Specifics

Y-axis and X-axis minimum/maximum, the number of grid divisions on each axis, axis label text, grid and label visibility toggles, update interval in milliseconds, horizontal step size, sample period, an optional “screen time” field that — when non-zero — automatically derives sample/update timing from a desired total visible time window, a trigger level, and a sweep/scroll graph mode.

Slider / Progress Bar Specifics

Minimum, maximum, and initial values, plus horizontal or vertical orientation.

Image Specifics

The source image file path, the generated C array name, and whether the image is resized to fill the widget while preserving its aspect ratio.

Rectangle Specifics

Corner radius in pixels (0 for sharp corners).

Changes in the property panel apply immediately and are reflected both on the canvas and in the live code preview.

9. Layout Tools

The Properties panel includes a set of layout utilities that act on the current selection:

- AutoFit Selected — resizes the selected widget to fit its current text and settings, useful after changing text size or content.
- Fit All To Display — clamps every widget on the current screen so that none extend past the display boundary, without changing widget positions more than necessary.
- Send Selected To Back / Bring Selected To Front — adjusts manual draw-order overrides so overlapping widgets (for example, a label sitting on top of a rectangle background) render in the desired stacking order.
- Align Left / Right / Top / Bottom — aligns all selected widgets to match the edge position of one reference widget in the selection.
- Center Horiz / Center Vert — centers selected widgets relative to each other or to the display, depending on selection.
- Distribute Horiz / Distribute Vert — spaces three or more selected widgets evenly along the chosen axis.

Widgets can also be nudged with the arrow keys once selected: a single press moves by 1 pixel, and holding Shift while pressing an arrow key moves by 10 pixels, which is useful for fine alignment without reaching for the mouse.

10. Off-Screen Widget Management

Because changing the display preset or rotation can leave previously well-placed widgets partially or fully outside the new display bounds, the designer provides dedicated tools to find and fix this:

- Find Off-Screen Widgets — scans the current screen and reports any widget that extends beyond the display boundary, without changing anything.
- Move Off-Screen Into View — repositions (clamps) any off-screen or partially clipped widgets back inside the display bounds.
- Delete Fully Off-Screen Widgets — removes widgets that are entirely outside the display area — typically left over after switching to a smaller display preset — without affecting widgets that are still partially visible.

The designer also performs an automatic safety check before export: if it detects widgets outside the display bounds, it asks for confirmation before exporting, since such widgets will not appear correctly on the physical screen.

11. Importing Images and Logos

The Import JPEG/BMP/PNG Logo button (and the Image / Logo Import widget type) loads a bitmap image from disk and converts it for embedded use.

When Pillow is installed, the designer can reliably load JPEG, BMP, and PNG files regardless of the host Qt installation's image plugins, and corrects image orientation based on embedded EXIF data where present. If Pillow is not available, image loading falls back to Qt's own image support, which may not cover all file variations.

At export time, each imported image is converted to a flat RGB565 pixel array (the standard 16-bit color format used by most SPI TFT controllers) and written into ui_images.c, with a matching extern declaration in ui_images.h. The Keep Aspect Ratio option determines whether the source image is stretched to exactly fill the widget's width and height, or scaled proportionally and padded with the widget's background color.

> *Because embedded RGB565 arrays consume two bytes per pixel, a 320×240 full-screen image will occupy roughly 150 KB of flash; keep imported images close to their final on-screen size to avoid unnecessarily large generated source files.*

12. Saving and Opening Projects

Save JSON writes the entire project — display settings, all screens, and every widget's full property set — to a single .json file. This is the designer's native project format and is the only format that can be reopened for further editing.

Open JSON loads a previously saved project file, replacing the current project in memory. Unsaved changes in the currently open project are not automatically preserved, so save before opening another file if you want to keep your current work.

New clears the current project and starts a fresh one with a single empty main screen, using the default display preset. As with Open, unsaved work is not automatically preserved.

A copy of the project JSON is also written automatically into the export folder every time you export firmware code (see Section 13), so the project state at the time of each export is always available alongside the generated C files.

13. Exporting Firmware Code

File → Export ui_screens.c/.h prompts for an output folder, then writes the following files into it:

- ui_screens.c and ui_screens.h — the screen enum, widget draw calls, touch handling dispatch, and any typed setter functions for Dynamic Text and Progress Bar widgets bound to firmware variables.
- ui_images.c and ui_images.h — only written if the project contains at least one Image widget; contains the RGB565 pixel arrays and their extern declarations.
- ui_display_config.h — defines the selected display name, width, height, rotation, and a driver identifier macro (for example TFT_DRIVER_ILI9341) for use by a universal SPI TFT driver layer.
- project.json — a snapshot of the full project at export time, in the same format used by Save JSON.
- A runtime notes text file — plain-text notes describing where to copy each generated file and any framework-level requirements that must be satisfied for the generated code to behave correctly (see Section 14).

Every generated .c and .h file (except project.json) carries a header comment identifying the designer version and the date and time it was generated, as a reminder that the file is auto-generated and will be overwritten by a future export rather than hand-edited.

Variable Bindings

For each Plot/Graph, Slider, or Progress Bar widget whose Source Type is not expression, the designer emits an extern declaration in ui_screens.h using the exact variable name you typed into Source Variable. Your firmware must define a global variable with that exact name and type somewhere in your project (typically in main.c) for the generated code to link successfully. Dynamic Text widgets are handled differently and do not get an extern declaration; see Setter Functions below.

Setter Functions

Dynamic Text and Progress Bar widgets additionally generate a UI_Set_<WidgetName>(tft, value) function, intended to be called from your firmware whenever the underlying value changes, so the displayed text or bar position is refreshed without redrawing the entire screen. For Dynamic Text, the value parameter type matches the widget's configured Source Type; for Progress Bar, it is always uint32_t regardless of Source Type. Progress Bar widgets are also extern-bound to their Source Variable (see Variable Bindings above), so calling the setter and keeping the bound global in sync are both expected; the bound global alone does not redraw the bar.

14. Integrating Generated Code into an STM32CubeIDE Project

Exporting from the designer only produces the screen-specific files described in Section 13 (ui_screens.c/.h, optionally ui_images.c/.h, and ui_display_config.h). Those files call into a shared driver layer that is not generated by the designer and must be added to your project separately, once, the first time you set up a new project. That driver layer ships as two folders:

- TFT-RelatedFiles — the universal, display-agnostic graphics and widget layer that every project needs regardless of which panel you're using. Its Inc/ folder contains tft_gfx.h, tft_widgets.h, tft_graph.h, tft_screen.h, tft_bus.h, tft_colors.h, tft_display.h, and fonts_5x7.h; its Src/ folder contains the matching .c files. Copy the entire contents of Inc/ into your project's Core/Inc and the entire contents of Src/ into Core/Src.
- DisplayDrivers — per-panel and per-touch-controller drivers, organized into subfolders: ILI9341-Resistive (ILI9341 plus an XPT2046 resistive touch driver and a touch_cal calibration helper), ILI9341-Capacitive (ILI9341 plus an FT6336U capacitive touch driver), ST7796-Capacitive (ST7796 plus FT6336U), ST7735, and ST7789. Copy only the one subfolder matching your actual hardware — copying more than one into the same project risks duplicate symbol errors, since more than one panel driver may define overlapping helper names.

With both pieces in place, complete the integration as follows:

1. Copy ui_screens.c (and ui_images.c, if present) into your project's Core/Src folder.
2. Copy ui_screens.h (and ui_images.h, if present) into your project's Core/Inc folder.
3. Copy ui_display_config.h into Core/Inc; it defines UI_TFT_DRIVER, UI_TFT_WIDTH, UI_TFT_HEIGHT, and UI_TFT_ROTATION, which tft_display.c reads to select the correct panel-specific register sequences and logical screen size at compile time.
4. If you haven't already, copy the TFT-RelatedFiles driver layer and the one matching DisplayDrivers subfolder into your project, as described above. This is a one-time setup per project, not a per-export step.

5. For each Plot/Graph, Slider, or Progress Bar widget's Source Variable field, define a matching global variable (correct name and type) somewhere in your firmware, typically in main.c. The designer emits an extern declaration for these three widget types in ui_screens.h; your firmware must define the real variable for the link step to succeed. Dynamic Text widgets are different: no extern is emitted for them, since their value is pushed in by your code calling a setter function rather than being read from a global (see step 7).
6. Call UI_Init(tft) once during startup, after your TFT driver itself has been initialized, where tft is a pointer to a TFT_GFX_t instance attached to the active panel driver.
7. Call UI_DrawScreen(tft, <screen enum value>) to draw a full screen. For Dynamic Text and Progress Bar widgets, call the generated UI_Set_<WidgetName>(tft, value) function whenever the underlying value changes, so the display updates incrementally instead of redrawing the whole screen; the parameter type matches the widget's configured Source Type (Dynamic Text) or is always uint32_t (Progress Bar). For Slider widgets, the touch handler updates the bound global directly — see step 8.
8. Route touch events into UI_ProcessTouch(tft, touched, x, y) so button, checkbox, radio, and slider widgets respond to input. Be aware that a plain Button widget's generated touch handler only emits a placeholder printf(...) call; it does not perform any action on its own, so you will need to add your own logic at that call site for the button to actually do something. Momentary Button widgets are different: if their label text contains “BACK,” “RETURN,” “NEXT,” or “PLOT,” the designer automatically generates a screen-navigation call instead of a placeholder.
9. If your project uses Plot/Graph widgets or any time-based widget behavior, call UI_UpdateLiveData(tft, now_ms) periodically (for example, from your main loop or a periodic timer callback) with a millisecond timestamp.

Dynamic Text Erase Height Requirement

> *The runtime notes file generated with every export calls out a specific requirement in your tft_widgets.c framework file: the function that updates a Dynamic Text or Progress/LabelValue widget's printed value must erase only the actual bitmap font height before redrawing, calculated as 8 * lv->text_size, not 16 * lv->text_size. Using the larger, incorrect height erases more vertical space than the text actually occupies and can overwrite the content of a nearby widget positioned just below it. Confirm this calculation in your framework before relying on Dynamic Text widgets in a new project.*

Image and Graph Framework Files

If your project uses Image widgets, keep only one copy of tft_graph.c/.h in the project to avoid duplicate symbol errors, and confirm the active project's tft_widgets.c is the corrected version referenced above, since both image rendering and dynamic text rely on functions in that file.

Stack and Heap Size — Read This Before Your First Build on a Larger Display

> *Larger, higher pixel-density displays (240×320 and up, and especially 480×320) drive correspondingly larger local buffers, frame/row buffers, and deeper call chains inside drawing and graph code than a small display does. If your project's linker script or CubeMX-generated startup configuration leaves the default (often quite small) minimum stack and heap sizes unchanged, the firmware can run seemingly fine for a while and then crash randomly — a classic stack overflow corrupting adjacent RAM, including the heap or other*

> *statically allocated data, rather than failing immediately or predictably. This kind of crash is difficult to diagnose because the failure point in the debugger is often far removed from the actual cause.*
>
> *Based on direct experience tracking down this exact failure mode, increase the Minimum Stack Size and Minimum Heap Size in your STM32CubeIDE project settings (or directly in the linker script / _Min_Stack_Size and _Min_Heap_Size symbols) generously before bringing up a new display, rather than leaving CubeMX defaults in place — particularly for the 320×240, 240×320, and 480×320 presets, where graph buffers and widget redraw paths use more stack than they do on the smaller 128×160/160×128/240×240 panels. There is no single correct number that fits every project, since actual usage depends on your own application code, RTOS configuration (if used), and how many widgets and graphs are active at once; size generously, then verify empirically using your toolchain's stack-usage reporting (e.g., GCC's -fstack-usage) or a watermarking/canary technique, and watch free heap over time if you use dynamic allocation anywhere in your application. Treat intermittent, hard-to-reproduce crashes — especially ones that appear only after adding a Plot/Graph widget or a larger display — as a prompt to revisit stack and heap sizing first.*

15. Minimal main.c Usage Example

This section walks through a small, self-contained example targeting the ST7796 480×320 display with FT6336U capacitive touch — the largest of the four supported presets, chosen here because it has enough room to show every technique in one screen. The same patterns apply to the ILI9341, ST7789, and ST7735 presets; only the UI_TFT_DRIVER / UI_TFT_WIDTH / UI_TFT_HEIGHT values in ui_display_config.h (generated automatically by the designer for whichever display you picked) and the touch driver you initialize change. The full file is provided alongside this manual as main_usage_example.c; the key pieces are walked through below.

The example assumes a project with three widgets already placed in the designer: a Dynamic Text widget named lblAdc (Source Variable adc_value, Source Type uint32_t), a Progress Bar widget named barAdc (Source Variable adc_raw, Source Type uint16_t), and a Button widget named btnStart.

Startup

Bring the physical panel up first, then attach the generic graphics context to it, then bring up touch, then initialize and draw the generated UI:

```
TFT_Display_Init();
TFT_GFX_AttachDriver(&tft, &TFT_Display_GFX_Driver);
TFT_GFX_SetRotation(&tft, UI_TFT_ROTATION);

/* FT6336U capacitive touch setup -- see the full example file for the
   complete FT6336U_Handle_t field assignments. */
FT6336U_Init(&touch);

UI_Init(&tft);
UI_DrawScreen(&tft, UI_SCREEN_MAIN);
HAL_ADC_Start(&hadc1);
```

Reading the ADC into a Dynamic Text Widget

Dynamic Text widgets have no bound global to maintain; push the value straight into the generated setter whenever it changes:

```
uint16_t raw = (uint16_t)HAL_ADC_GetValue(&hadc1);
adc_value = (uint32_t)raw;       /* ordinary application variable, not an extern */
UI_Set_lblAdc(&tft, adc_value);  /* this call updates the screen */
```

Driving a Progress Bar Widget From the Same Reading

Progress Bar widgets are different: the designer emits extern uint16_t adc_raw; in ui_screens.h, so your firmware must define that exact global, but writing to it alone does not redraw the bar — you still need to call the setter:

```
adc_raw = raw;                 /* satisfies the generated extern declaration */
UI_Set_barAdc(&tft, adc_raw);  /* actually redraws the bar on screen */
```

Handling Touch

Read the touch controller, then forward whatever it reports into UI_ProcessTouch() every loop iteration, regardless of whether a touch is currently active — the generated code needs to see both the press and the release to track button state correctly:

```
FT6336U_TouchData_t data;
FT6336U_ReadTouch(&touch, &data);

uint8_t touched = (data.touches > 0) ? 1U : 0U;
uint16_t tx = touched ? data.p1.x : 0U;
uint16_t ty = touched ? data.p1.y : 0U;

UI_ProcessTouch(&tft, touched, tx, ty);
```

For the resistive ILI9341 preset, replace the FT6336U calls with XPT2046_GetPoint() and use its .touched, .x, and .y fields the same way; the call into UI_ProcessTouch() itself doesn't change.

> *The btnStart button won't do anything yet. This is expected — see the “regular Button widget doesn't do anything when pressed” entry in Section 16. Open the generated ui_screens.c, find the TFT_Button_Update(tft, &ui_btnStart, ...) block, and add your own action where the placeholder printf() call currently sits.*

Main Loop

Poll touch every iteration; throttle the ADC read so you're not redrawing faster than necessary; call UI_UpdateLiveData() every iteration so any Plot/Graph widgets stay current:

```
while (1)
{
    App_PollTouchAndUpdateUI();
```

```
    if ((HAL_GetTick() - last_adc_poll_ms) >= 50U)
    {
        last_adc_poll_ms = HAL_GetTick();
        App_ReadAdcAndUpdateUI();
    }

    UI_UpdateLiveData(&tft, HAL_GetTick());
}
```

This example was checked for correctness by generating real ui_screens.c/.h output from the designer for these exact three widgets and compiling the resulting calls against the actual tft_gfx.h, tft_widgets.h, and ft6336u.h headers; every function name, parameter order, and type shown above matches what the designer and driver layer actually expect, not an approximation.

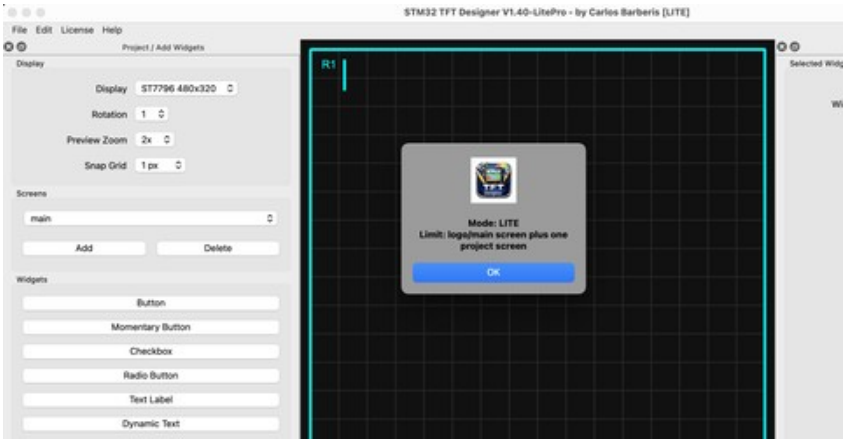
16. Licensing: Lite and Pro Modes

The application ships as a single installer that runs in Lite mode by default. Lite mode is fully functional for evaluating the designer and building single-screen projects at no cost. Upgrading to Pro mode removes the screen limit and is activated by entering a license key tied to your email address.

Lite Mode

When no valid license is present the application starts in Lite mode. The title bar displays [LITE] and a startup dialog confirms the active mode and its limitation:

Mode: LITE — logo/main screen plus one project screen.



Startup dialog confirming Lite mode and its single-project-screen limit

All widget types, all display presets, all layout and alignment tools, the Edit menu, code export, and the full Properties panel are available in Lite mode. The only restriction is the number of project screens: one project screen in addition to the optional logo/main screen.

Pro Mode

Pro mode removes the screen limit entirely, allowing as many screens as your project requires. All other functionality is identical to Lite mode — Pro simply unlocks unlimited screens.

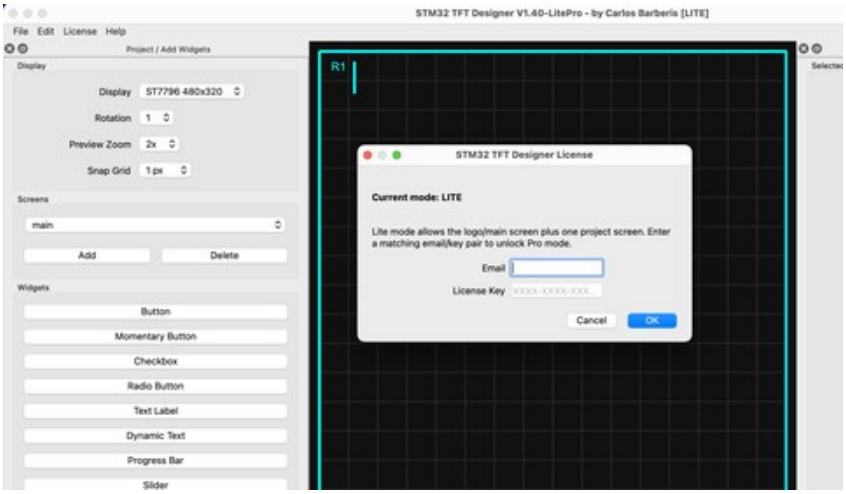
To purchase a Pro license, contact carlos@bartek.com or visit the product page. Provide the email address you want the license tied to. You will receive a license key in the format XXXX-XXXX-XXX... by return email.

> *The license is tied to your email address. Keep both your email and your license key in a safe place — you will need them if you reinstall the application or move to a new machine.*

Entering Your License Key

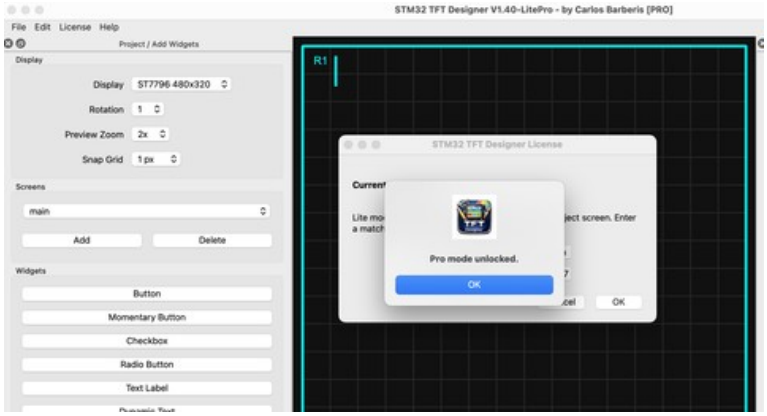
Once you have received your license key, activate Pro mode as follows:

- Open the application. It will start in Lite mode.
- From the menu bar, select License → Enter License Key…
- The STM32 Designer License dialog will open, showing the current mode (LITE) and a brief description of the Lite restriction.



The Enter License Key dialog — enter your registered email and the key exactly as provided

- Type your registered email address in the Email field.
- Type or paste your license key in the License Key field (format: XXXX-XXXX-XXX…). The key is case-sensitive.
- Click OK.
- If the email and key match, a confirmation dialog appears: “Pro mode unlocked.” Click OK to dismiss it.



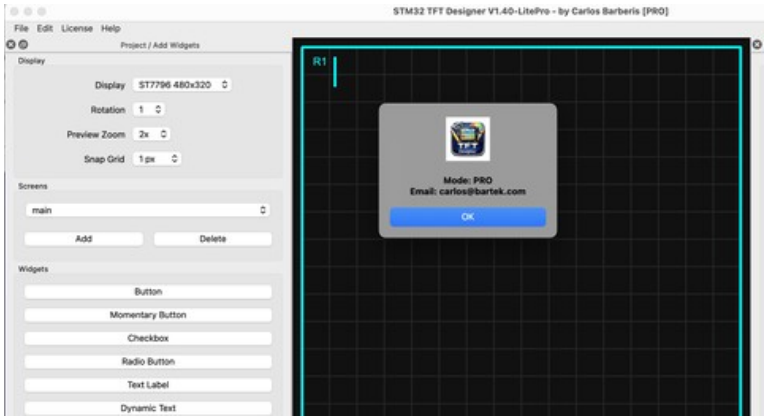
Pro mode unlocked confirmation — appears immediately after a valid key is accepted

- The title bar updates to [PRO] and the screen limit is removed immediately. No restart is required.

> *If the key is rejected, check that the email address matches exactly (including capitalisation) and that the license key was copied in full with no extra spaces. Contact carlos@bartek.com if the problem persists.*

Checking Your License Status

To confirm which mode is active at any time, select License → License Status from the menu bar. In Lite mode the dialog shows Mode: LITE and the screen restriction. In Pro mode it shows Mode: PRO and the registered email address.



License Status dialog confirming Pro mode with the registered email address

Pricing

Both tools are available individually or as a discounted bundle. All licenses are one-time purchases — no subscription, no annual renewal.

- STM32 OLED Designer 128×64 — $35.00
- STM32 TFT Designer — $49.00
- Bundle (both tools) — $75.00

To purchase, contact carlos@bartek.com with the email address you want the license tied to and the tool(s) you require. A license key will be sent by return email.

17. Troubleshooting

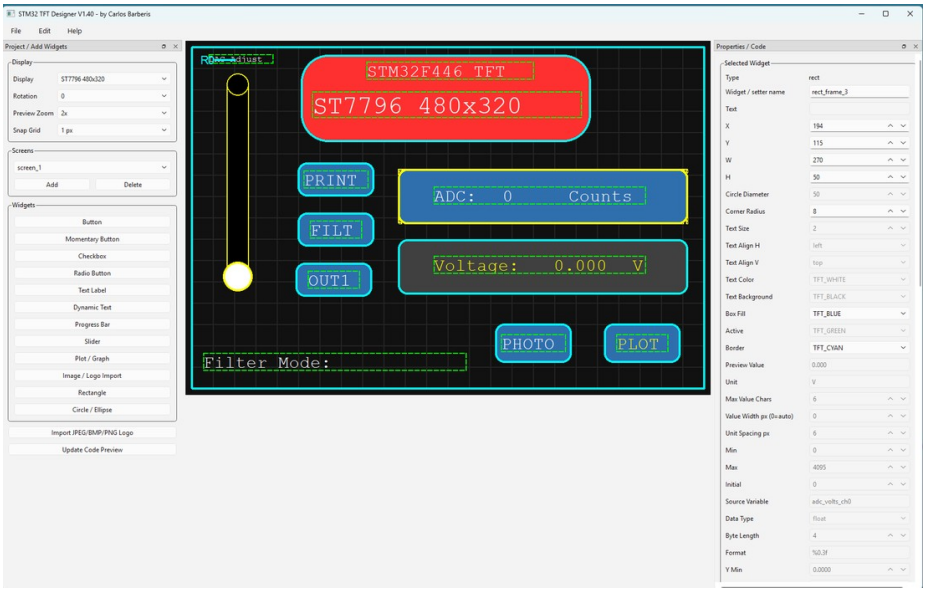
Symptom	Likely Cause / Fix
A widget doesn't appear after exporting and flashing.	Re-open the project in the designer and use Find Off-Screen Widgets on the relevant screen; the most common cause is a widget positioned outside the active display's bounds for the currently selected rotation.
Importing an image fails or shows a “Could not open the selected image” message.	Install Pillow (pip install pillow) for the most reliable cross-format image loading, particularly for JPEG and some BMP variants that may not be supported by the Qt image plugins bundled with the application.
Dynamic Text values look correct briefly, then show overlapping or garbled digits.	This is almost always the tft_widgets.c erase-height issue described in Section 14; verify the dynamic-text erase rectangle uses 8 * lv->text_size.
The generated header declares an extern variable my project doesn't have.	Check the Source Variable field of every Plot/Graph, Slider, and Progress Bar widget on every screen; each one must match a real global variable defined in your firmware, with a compatible type. Dynamic Text widgets do not generate an extern declaration, since their value is pushed in via a UI_Set_<WidgetName>() call rather than read from a global.
Project file won't open / “Open failed” message.	The JSON file may have been edited by hand and is no longer valid JSON, or it was saved by an incompatible future version of the designer. Try opening a known-good backup, or inspect the file's JSON syntax directly.
Firmware crashes randomly, freezes, or behaves erratically — especially after adding a Plot/Graph widget or moving to a larger display.	Check your project's minimum stack and heap size settings before looking anywhere else. This is one of the most common and hardest-to-diagnose issues when driving larger displays such as 240×320 or 480×320; see the stack/heap sizing guidance in Section 14.
A regular Button widget doesn't do anything when pressed.	This is expected, not a bug: a plain Button's generated touch handler only contains a placeholder printf(...) call. Open ui_screens.c, find the TFT_Button_Update(...) block for that button, and add your own logic there. For automatic screen navigation, consider a Momentary Button instead, since the designer auto-generates a navigation call for those when the label text contains “BACK,” “RETURN,” “NEXT,” or “PLOT.”
Build fails with an “undeclared identifier” error mentioning TFT_LIGHTGREY.	This happens with older copies of tft_colors.h that don't define TFT_LIGHTGREY, even though it's offered in the designer's color dropdown. Update tft_colors.h to add #define TFT_LIGHTGREY 0xD69A (the current library includes this), or change the affected widget's color to TFT_DARKGREY or another defined color in the designer and re-export.

18. Example Projects on Real Hardware

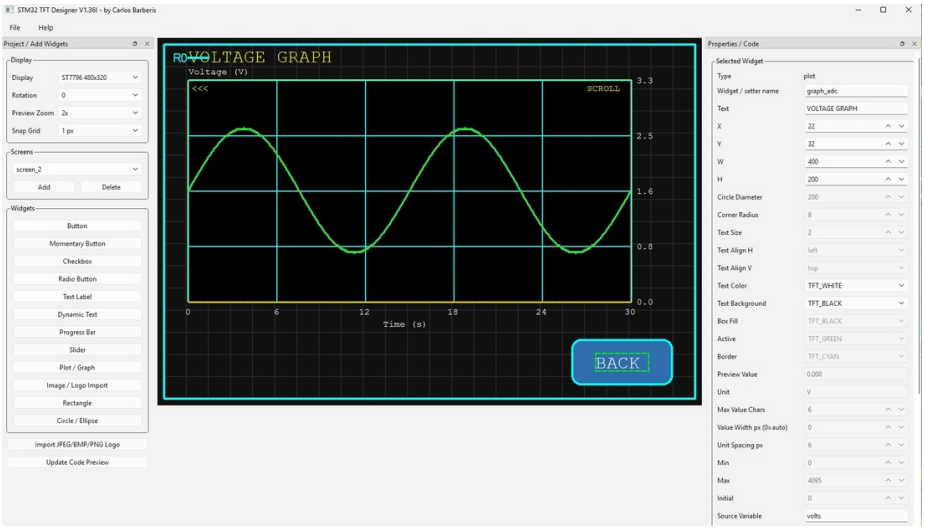
The screens below were designed in STM32 TFT Designer and exported to C, then built and flashed onto real STM32F446 hardware driving three different display sizes, showing the same workflow scaling from a small 1.54″ panel up to a 480×320 display.

The Designer in Use

The two screenshots below show the application itself mid-project: the canvas with widgets selected (showing the 480×320 project's screen_1), and a Plot / Graph widget's full property set populated in the right-hand panel (screen_2), both for the same 480×320 project shown running on real hardware further below.



The STM32 TFT Designer V1.40 canvas for the 480×320 project, showing the new Edit menu between File and Help in the menu bar

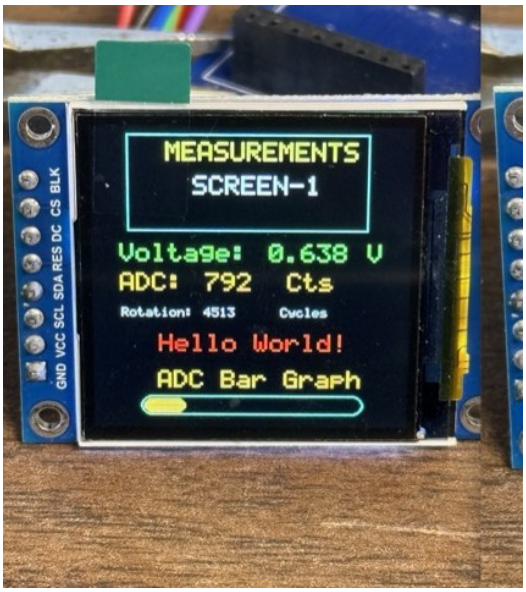


A Plot / Graph widget selected on “screen_2,” showing axis ranges, divisions, and the bound Source Variable in the Properties panel

240×240, 1.54″ IPS Panel

A compact display well suited to simple measurement readouts and status screens where board space is limited.

Splash / logo screen on the 1.54" 240×240 display



Live measurement screen with voltage, ADC counts, and an ADC bar graph

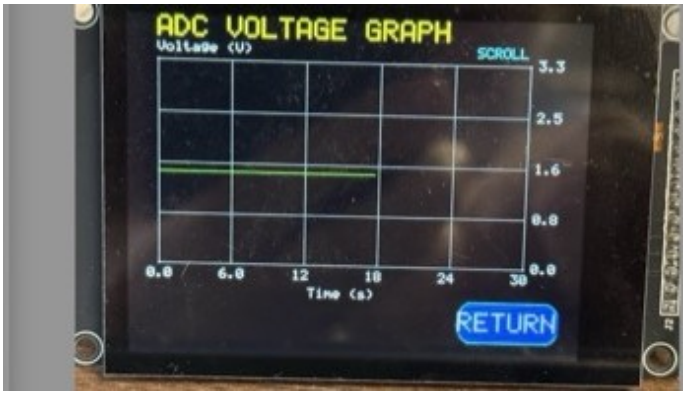
320×240, ILI9341-Class Panel

A common mid-size touch display, shown here running a multi-screen project with a main menu, a controls screen with checkboxes and radio buttons, a live ADC/DAC readout with a plot, and an image widget displaying an imported photo.

Main menu screen with Hello / Bye / Stay buttons and screen navigation

Controls screen with Enable/Disable checkboxes and Mode1/Mode2 radio buttons

ADC/DAC readout screen with a vertical bar and a DAC output slider

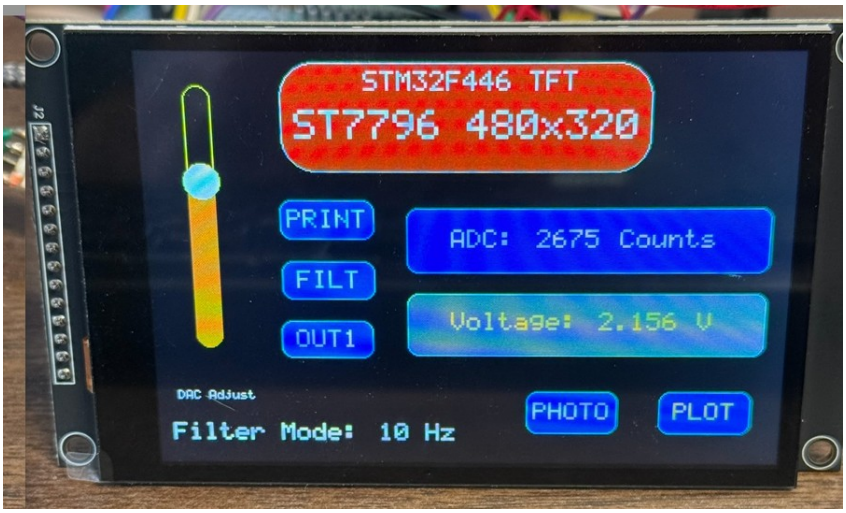


Scrolling voltage-vs-time plot with axis labels and gridlines

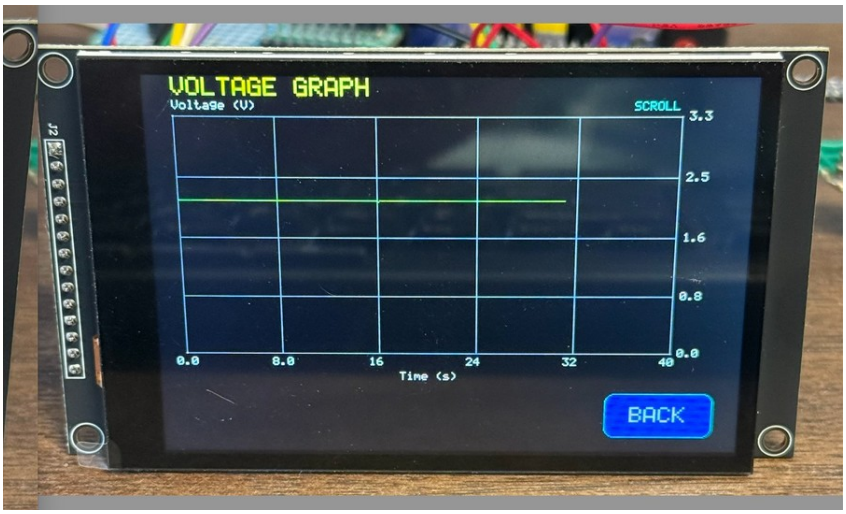
Imported photo displayed via the Image / Logo Import widget

480×320, ST7796 Panel

The largest supported preset, with more room for simultaneous sliders, status fields, and a live plot on one screen.



ST7796 480x320 display name banner alongside a slider, ADC counts, and voltage readout



Scrolling voltage-vs-time plot on the 480x320 display

Imported photo displayed at larger scale on the 480x320 display

> *Each of these projects was built and integrated following the workflow in Sections 13 and 14 of this manual, including the dynamic-text erase-height fix and the stack/heap sizing guidance — both of which become more relevant as display size and widget count increase.*

19. Appendix: Keyboard Shortcuts

Action	Shortcut
Copy selected widget(s)	Ctrl+C
Paste widget(s) from clipboard	Ctrl+V
Duplicate selected widget(s)	Ctrl+D
Delete selected widget	Del
Nudge selected widget by 1 pixel	Arrow keys
Nudge selected widget by 10 pixels	Shift + Arrow keys

This manual describes STM32 TFT Designer V1.40-LitePro. Screenshots and field names refer to the application as shipped; minor wording in dialogs may vary slightly between point releases.

© 2026 Carlos Barberis. All rights reserved.